

Uruchamianie aplikacji w Kubernetes

kubectl • YAML • Pod • Deployment • Service

```
kubectl apply -f deployment.yaml
```



Cel modułu

- uruchomić pierwszą aplikację
- zobaczyć różnicę między Podem, Deploymentem i Service
- wygenerować manifest YAML z komendy kubectl
- nauczyć się bezpiecznie wklejać manifesty w vi
- sprawdzić stan aplikacji podstawowymi komendami



kubectl w praktyce

get

lista obiektów
`kubectl get pods`

describe

szczegóły i eventy
`kubectl describe pod`

logs

logi aplikacji
`kubectl logs`

apply

tworzenie /
aktualizacja
`kubectl apply -f`

delete

sprzątanie
`kubectl delete -f`



CLI → YAML

Najlepszy sposób na naukę manifestów:

```
kubectl create ... --dry-run=client -o yaml
```

Komenda nie tworzy obiektu — tylko pokazuje YAML, który można zapisać do pliku.

```
... --dry-run=client -o yaml > manifest.yaml
```



YAML — minimum potrzebne w Kubernetes

pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: qa-web
spec:
  containers:
    - name: nginx
      image: nginx:1.27
```

- wcięcia mają znaczenie
- spacje zamiast tabów
- listy zaczynają się od '-'
- każdy manifest ma:
 - apiVersion
 - kind
 - metadata
 - spec



vi: bezpieczne wklejanie manifestu

Przy wklejaniu YAML-a do vi najpierw włącz tryb paste.

```
vi pod.yaml  
  
:set paste  
i  
# teraz wklejamy manifest  
Esc  
:wq
```

To chroni przed automatycznymi wcięciami, które potrafią zepsuć YAML.



Pierwszy Pod — z komendy

```
kubectl run qa-web --image=nginx:1.27  
kubectl get pods  
kubectl describe pod qa-web
```

Pod to najmniejsza jednostka uruchomieniowa w Kubernetes.



Pierwszy Pod — YAML

```
kubectl run qa-web \
  --image=nginx:1.27 \
  --dry-run=client -o yaml >
pod.yaml
```

```
vi pod.yaml
kubectl apply -f pod.yaml
```

sprawdzenie

```
kubectl get pod qa-web
kubectl delete pod qa-web
kubectl apply -f pod.yaml
```



Deployment

Deployment zarządza Podami i pilnuje, żeby aplikacja działała.

```
kubectl create deployment qa-web --image=nginx:1.27
```

```
kubectl get deployments  
kubectl get pods
```



Deployment — manifest z kubectl

```
kubectl create deployment qa-web \  
  --image=nginx:1.27 \  
  --dry-run=client -o yaml > deployment.yaml
```

Edytujemy YAML, a potem stosujemy zmianę w klastrze.

```
vi deployment.yaml  
kubectl apply -f deployment.yaml
```



Skalowanie aplikacji

```
kubectl scale deployment qa-web --replicas=3  
kubectl get pods
```

Deployment tworzy tyle Podów, ile zadeklarujemy w replicas.

Pod

Pod

Pod



Pod Management

```
kubectl get pods  
kubectl delete pod <pod-name>  
kubectl get pods
```

Jeśli Pod zniknie, Deployment utworzy nowy.

To jest podstawowy mechanizm self-healing w Kubernetes.



Service

Service daje stały adres dostępu do Podów.

```
kubectl expose deployment qa-web \  
  --port=80 \  
  --target-port=80
```

port = port Service
targetPort = port kontenera/Poda, do którego Service przekazuje ruch

```
kubectl get svc  
kubectl describe svc qa-web
```



Dostęp do aplikacji: port-forward

```
kubectl port-forward svc/qa-web 8080:80  
curl http://localhost:8080
```

port-forward jest najprostszą metodą sprawdzenia aplikacji bez Ingressa i LoadBalancer.



Workflow QA: sprawdzenie aplikacji

```
kubectl get pods  
kubectl get svc  
kubectl describe pod <pod-name>  
kubectl logs <pod-name>  
kubectl port-forward svc/qa-web 8080:80
```

Najpierw status. Potem szczegóły. Na końcu logi i test dostępu.



Lab: flow manifestami YAML

Manifesty generujemy przez dry-run,
zapisujemy do plików
i używamy przez kubectl apply.

```
kubectl create deployment qa-web \  
  --image=nginx:1.27 \  
  --dry-run=client -o yaml >  
deployment.yaml
```

```
kubectl apply -f deployment.yaml
```

Manifesty w repo:

- pod.yaml
- deployment.yaml
- service.yaml



Lab: flow komendami kubectl

```
kubectl run qa-web --image=nginx:1.27  
kubectl create deployment qa-web --image=nginx:1.27  
kubectl scale deployment qa-web --replicas=3  
kubectl expose deployment qa-web --port=80 --target-port=80  
kubectl port-forward svc/qa-web 8080:80
```

Najpierw pokazujemy szybkie tworzenie obiektów z terminala.

